

Como um agente passa de responder para agir?

Encontro 04 · Sistemas Multiagentes / Agentic AI

O limite que ficou do Encontro 03

CampusBot conversa ✓
CampusBot mantém um ciclo ✓
CampusBot consulta equipamentos ?
CampusBot abre chamados ?

Uma resposta pode abrir um chamado?

Execute antes de explicar

```
uv run python praticas/encontro-04/01_so_responde.py --offline
```

“O projetor do Lab 4 não funciona. Abra um chamado.”

O modelo pode dizer algo plausível

```
"Eu abriria um chamado para o projetor do Lab 4."
```

Mas o terminal também diz:

```
Nenhum ambiente foi consultado ou modificado.
```

O que falta entre dizer e fazer?

Texto $\neq$ consequência no ambiente

A menor mudança possível

```
abrir_chamado(local, problema)
```

Primeiro: ela funciona sem LLM.

Execute a ferramenta sozinha

```
uv run python praticas/encontro-04/02_primeira_ferramenta.py
```

Preveja o ambiente antes e depois.

O que a execução mostra

```
ambiente antes: []  
                ↓  
abrir_chamado(...)  
                ↓  
ambiente depois: [CH-001]
```

Quem abriu o chamado?

Duas responsabilidades, por enquanto

Modelo

escolhe ou solicita

Ferramenta

executa uma capacidade

Nesta execução não havia modelo. A ferramenta, chamada pelo programa, mudou o ambiente.

Vamos conectá-los sem esconder a fronteira

```
modelo solicita uma ferramenta
      ↓
Python recebe nome + argumentos
      ↓
Python chama a função
```

O contrato da decisão continua pequeno

```
{"tipo": "usar_ferramenta",  
  "ferramenta": "abrir_chamado",  
  "argumentos": {"local": "Lab 4", "problema": "projektor não funciona"}}
```

ou

```
{"tipo": "responder", "mensagem": "0 chamado CH-001 foi aberto."}
```

**E se antes precisamos descobrir
se já há um chamado?**

O pedido muda

“Veja se já existe um chamado. Se não existir, abra um.”

Agora existem duas ferramentas:

```
consultar_chamados(local)
abrir_chamado(local, problema)
```

Faça uma previsão da trajetória

```
uv run python praticas/encontro-04/03_agente_com_ferramentas.py --offline
```

Qual ferramenta vem primeiro? O que a próxima decisão fará se a consulta estiver vazia?

Passo 1: o modelo solicita

```
[MODELO]  
Quero usar: consultar_chamados  
Argumentos: {local: Lab 4}
```

Passo 1: o programa executa

```
[CONTROLADOR] Executando ferramenta...  
[FERRAMENTA] consultar_chamados(...)  
[RESULTADO] {ok: true, chamados: []}
```

Passo 2: uma nova decisão

```
[MODELO]
```

```
Quero usar: abrir_chamado
```

A segunda decisão usa uma informação que não existia antes da primeira ferramenta.

Passo 2: o ambiente muda

```
antes: []  
depois: [CH-001]
```

```
[RESULTADO] {ok: true, protocolo: CH-001}
```

Passo 3: o modelo responde

`"O chamado CH-001 foi aberto."`

O ciclo que acabamos de observar

```
decidir  
→ agir  
→ observar  
→ decidir novamente
```

Quem escreveu antecipadamente

“consultar → abrir → responder”?

Uma resposta curta

```
controlador: escreve o ciclo  
modelo: escolhe o próximo passo  
ferramenta: produz o resultado
```

Em um workflow, a sequência poderia estar escrita de antemão. Aqui, ela é escolhida durante a execução a partir da observação.

Mude o ambiente

```
uv run python praticas/encontro-04/03_agente_com_ferramentas.py \  
  --com-chamado-existente --offline
```

Preveja: ele abrirá outro chamado?

Com CH-001 já presente

```
modelo → consultar_chamados  
resultado → CH-001 existe  
modelo → responder
```

A ferramenta disponível é a mesma. O que mudou foi a observação.

**E se a ferramenta executa,
mas seu resultado não volta?**

Corte uma passagem de propósito

```
uv run python praticas/encontro-04/03_agente_com_ferramentas.py \  
  --com-chamado-existente --resultado-nao-volta --offline
```

O sintoma

```
consultar_chamados executou ✓  
CH-001 existe no ambiente ✓  
resultado chegou ao modelo ✕
```

```
modelo pede consultar_chamados novamente
```

**O resultado da ferramenta
vira informação para a próxima decisão.**

Pedir uma ação prova que ela deu certo?

Faça a abertura falhar

```
uv run python praticas/encontro-04/04_quando_a_ferramenta_falha.py --offline
```

A ferramenta devolve uma evidência diferente

```
{"ok": false, "erro": "serviço indisponível"}
```

O modelo recebe essa evidência e responde que não conseguiu abrir o chamado.

Pedir uma ação

≠ executar uma ação

≠ a ação ter sucesso

Quatro responsabilidades que a experiência pediu

Parte	Pergunta que responde
Modelo	Qual é o próximo passo?
Ferramenta	Que capacidade o programa pode executar?
Ambiente	O que foi consultado ou mudou?
Controlador	Quem liga decisão, execução e nova observação?

Leia a arquitetura no código

```
modelo.py → decisão  
controlador.py → execução + ciclo  
ferramentas.py → capacidade  
ambiente.py → chamados em memória
```

A LLM escolhe.

O programa executa.

A ferramenta produz uma consequência no ambiente.

O resultado volta para a próxima decisão.

Missão autônoma — uma ferramenta a mais

```
consultar_status(protocolo)
```

Produza uma execução com:

```
decisão → solicitação → execução → resultado → decisão seguinte
```

**Quando uma LLM pede uma ferramenta,
quem realmente a executa?**